

Product Requirements Documents

Three Flagship Portfolio Projects — Built on the Engineering Project Blueprint (v1.0)

Each PRD is scoped to be buildable solo, in a few focused weeks, while forcing you to implement — not just read about — the specific blueprint patterns called out per project. Each follows the same structure so they're easy to execute against in sequence.

PRD 1 — Multi-Tenant SaaS Invoicing & Billing Platform

1. Overview

A billing platform for small businesses to manage customers, issue recurring invoices, accept payments, and track balances in real time — with hard tenant isolation, since this is a multi-tenant SaaS product from day one.

2. Problem Statement

Small businesses currently juggle spreadsheets, generic invoicing tools, and manual payment tracking. They need one system where creating a customer, sending a recurring invoice, collecting payment, and seeing an up-to-date balance is fast and reliable — and where their data can never leak across tenant boundaries.

3. Goals

Prove correct multi-tenant data isolation under test, not just by convention.
Handle payment-provider webhooks safely under retries and duplicate delivery.
Generate and deliver invoices/receipts asynchronously without blocking user-facing requests.
Provide a real-time, accurate balance view per customer and per tenant.

4. Non-Goals

Multi-currency accounting or tax-jurisdiction logic (flag as a documented limitation).
Full double-entry bookkeeping / general ledger.
Custom invoice template designer (use one fixed, clean template).

5. Users & Personas

Persona	Needs
Business owner (tenant admin)	Create customers, send invoices, see balances, manage team access

Persona	Needs
Staff user (tenant member)	Create/view invoices and customers within permission scope
End customer (payer)	View and pay an invoice via a hosted payment link, no login required

6. Functional Requirements

6.1 Tenancy & Access

FR1: Every account belongs to exactly one tenant (business). Users belong to a tenant with a role (`owner`, `staff`).

FR2: All domain queries are scoped by `tenant_id`; no query path may return cross-tenant data.

FR3: Role-based authorization: `owner` can manage billing/team; `staff` can create/view invoices only.

6.2 Customers

FR4: Create, edit, list, and archive customers within a tenant.

FR5: Each customer has a running balance derived from invoice and payment history (not a manually edited field).

6.3 Invoicing

FR6: Create one-off and recurring invoices (weekly/monthly/custom interval) for a customer.

FR7: Recurring invoices auto-generate on schedule via a background job, not a request-triggered action.

FR8: Each invoice has a lifecycle: `draft → sent → paid | overdue | void`.

FR9: Invoice PDF is generated asynchronously and attached to the sent email.

6.4 Payments

FR10: Customers pay via a hosted, tokenized payment link — no card data touches our servers directly.

FR11: Payment-provider webhooks update invoice status; duplicate webhook deliveries must not double-apply a payment.

FR12: On successful payment, a receipt is generated and emailed asynchronously (outbox → worker).

6.5 Balance & Reporting

FR13: Tenant dashboard shows real-time aggregate balance (outstanding, overdue, paid this month).

FR14: Customer detail page shows full invoice/payment history.

7. Non-Functional Requirements

Category	Requirement
Isolation	Automated tests proving cross-tenant read/write denial (§14.2 of blueprint)
Idempotency	Payment webhook processing is idempotent under at-least-once delivery
Latency	Dashboard balance query p95 < 300ms at 10K invoices/tenant

Category	Requirement
Reliability	Receipt/PDF generation retried on failure with bounded backoff; failures visible in a dashboard
Security	Payment provider secrets centralized; no card data logged or stored
Rate limiting	Payment endpoint rate-limited per tenant to prevent abuse

8. Data Model (core entities)

`tenants` (id, name, plan)
`users` (id, tenant_id, email, role)
`customers` (id, tenant_id, name, email)
`invoices` (id, tenant_id, customer_id, status, amount, due_date, recurrence_rule)
`payments` (id, tenant_id, invoice_id, provider_payment_id UNIQUE, amount, status)
`outbox_events` (id, tenant_id, type, payload, processed_at)

Unique constraint on `payments.provider_payment_id` is the idempotency guard against duplicate webhook processing.

9. Architecture Notes (per blueprint)

NestJS modules: `tenants`, `customers`, `billing` (invoices), `payments` (provider adapter), `notifications`.
Payments module wraps the provider SDK behind a typed adapter interface (timeout, retry, idempotency key, structured telemetry — §7.5).
Invoice-paid event written to `outbox_events` in the same transaction as the payment status update (§8.5.1), then published to a BullMQ worker that sends the receipt.
Redis used only for rate-limiting the payment endpoint — not as a source of truth (§9.2 caching decision framework: concrete use case required).

10. API Surface (representative)

```

□
POST /customers
GET /customers/:id
POST /invoices
POST /invoices/:id/send
POST /webhooks/payments (provider-authenticated, idempotent)
GET /dashboard/balance

```

11. Milestones

Walking skeleton: one tenant, one customer, one manual invoice, one manual "mark paid" action, deployed.
Recurring invoice generation + async PDF/email pipeline.
Real payment-provider integration (sandbox) + idempotent webhook handling.

Multi-tenant isolation test suite + cross-tenant denial tests.

Dashboard balance view + load test at 10K invoices/tenant.

12. Success Metrics (for resume/portfolio)

Cross-tenant isolation: 100% of isolation test suite passing, documented in README.

Webhook idempotency: 0 duplicate-payment incidents across N simulated duplicate deliveries.

Async pipeline: receipt delivered within X seconds p95 of payment confirmation.

13. Open Risks

Payment-provider sandbox limitations may not fully replicate production webhook retry behavior — mitigate with a manual duplicate-delivery test harness.

PRD 2 — Marketplace Order & Fulfillment System

1. Overview

A multi-vendor marketplace backend where buyers place orders against a shared catalog, inventory decrements correctly under concurrent checkout load, and fulfillment status stays in sync with (simulated) vendor systems.

2. Problem Statement

Marketplace checkout is a classic concurrency correctness problem: without careful design, concurrent buyers can oversell limited stock, and vendor-side fulfillment updates can arrive out of order or get lost. This project proves the concurrency and reconciliation logic actually works, not just that it compiles.

3. Goals

Guarantee inventory never goes negative under concurrent checkout load.

Keep order state and vendor fulfillment state reconciled even when vendor sync calls fail or complete late.

Demonstrate an expand-and-contract migration executed mid-project without downtime.

4. Non-Goals

Real payment processing (reuse a stub/mock payment step; focus is on inventory/fulfillment correctness).

Vendor onboarding UI (vendors and their inventory can be seeded via fixtures/admin script).

Recommendation/search ranking.

5. Users & Personas

Persona	Needs
Buyer	Browse catalog, check out, track order/fulfillment status
Vendor (simulated)	Inventory and fulfillment updates arrive via a mock external API
Platform admin	Investigate and manually resolve fulfillment discrepancies

6. Functional Requirements

6.1 Catalog & Inventory

FR1: Products belong to a vendor and have a stock count.

FR2: Inventory decrements only occur inside a transaction that also creates the order line item.

FR3: Two concurrent checkouts for the last unit of stock must result in exactly one success and one clear failure — never both succeeding.

6.2 Checkout & Orders

FR4: An order is created with line items across potentially multiple vendors.

FR5: Order has a lifecycle: `placed → confirmed → fulfilling → fulfilled | cancelled`.

FR6: Checkout is a single atomic operation from the buyer's perspective even though it spans multiple vendor line items.

6.3 Vendor Fulfillment Sync

FR7: A background job calls a (simulated) vendor API per order line item to request fulfillment.

FR8: Vendor sync calls have bounded retries with backoff; permanently failed line items land in a dead-letter state visible to admins.

FR9: A reconciliation job periodically re-checks "in-flight" vendor calls whose result is ambiguous (timeout without confirmed response) and resolves them (§7.5 — do not assume a timeout means failure).

6.4 Admin

FR10: Admin view lists orders with reconciliation status and allows manual resolution of stuck/ambiguous fulfillment states.

7. Non-Functional Requirements

Category	Requirement
Correctness	Zero overselling under load test with concurrent checkout simulation
Reliability	Vendor sync worker uses bounded concurrency; one slow vendor cannot starve others
Auditability	Every inventory decrement and fulfillment state change is logged with correlation ID

Category	Requirement
Migration safety	Order-state model change executed via expand-and-contract with zero downtime, demonstrated and documented
Observability	Trace correlates checkout → inventory decrement → vendor sync → fulfillment confirmation

8. Data Model (core entities)

`vendors` (id, name)
`products` (id, vendor_id, name, stock_count)
`orders` (id, buyer_id, status)
`order_line_items` (id, order_id, product_id, vendor_id, quantity, fulfillment_status)
`vendor_sync_jobs` (id, order_line_item_id, status, attempts, last_attempted_at)

9. Architecture Notes (per blueprint)

NestJS modules: `catalog`, `orders`, `inventory`, `fulfillment`, `integrations/vendor-mock`.

Inventory decrement + order-line creation wrapped in one PostgreSQL transaction with row-level locking (`SELECT ... FOR UPDATE`) on the product row (§8.5).

Vendor sync implemented as a BullMQ worker with per-vendor concurrency limits (§9.4) and dead-letter queue for exhausted retries.

Reconciliation job (scheduled, §9.5) resolves ambiguous in-flight vendor calls rather than assuming failure.

Order-state model migration executed mid-project using expand-and-contract (§8.3): add new status column → dual-write → backfill → cut over reads → drop old column in a later release, with before/after evidence of zero request errors.

10. API Surface (representative)

```

□
GET  /catalog
POST /checkout      (atomic across vendor line items)
GET  /orders/:id
GET  /admin/orders?status=stuck
POST /admin/orders/:id/resolve

```

11. Milestones

Walking skeleton: single-vendor catalog, checkout, no concurrency handling yet.

Concurrency-safe inventory decrement + load test proving no overselling.

Vendor sync worker with retries, dead-letter handling, and reconciliation job.

Admin console for stuck-order resolution.

Mid-project expand-and-contract migration on the order-state model, executed and documented live.

12. Success Metrics (for resume/portfolio)

0 overselling incidents across a load test of N concurrent checkouts against limited stock.
Vendor sync dead-letter rate and mean time-to-reconciliation reported from real job data.
Documented zero-downtime migration with before/after error-rate evidence.

13. Open Risks

Simulated vendor API needs deliberately injected failure modes (timeouts, slow responses, duplicate confirmations) to make the reconciliation logic meaningfully testable.

PRD 3 — Appointment Scheduling Platform with Timezone-Safe Booking

1. Overview

A scheduling platform where service providers publish availability and clients book slots across timezones, with zero double-booking, automated reminders, and a cancellation window — the classic “looks simple, is actually a hard concurrency and timezone problem” build.

2. Problem Statement

Naive booking systems double-book slots under concurrent requests and mishandle timezones (off-by-one-hour bugs, DST transitions). This project builds a booking core that is provably correct under both dimensions, plus the operational layer (reminders, cancellations) around it.

3. Goals

Guarantee no double-booking under concurrent booking attempts for the same slot.
Store and display times correctly across timezones, including DST transitions.
Deliver reminders reliably at the correct scheduled offset, once and only once.

4. Non-Goals

Payment collection for bookings.
Complex recurring-availability rules beyond a simple weekly pattern.
Provider discovery/marketplace features (single-provider or fixed provider list is fine).

5. Users & Personas

Persona	Needs
Service provider	Publish weekly availability, view upcoming bookings, set cancellation window
Client	Browse available slots in their own timezone, book, cancel within policy
System	Send reminders and cancellation-window enforcement automatically

6. Functional Requirements

6.1 Availability

FR1: Providers define weekly recurring availability windows in their own timezone.

FR2: Availability is materialized into concrete bookable slots for a rolling future window (e.g., next 60 days).

6.2 Booking

FR3: Clients view available slots converted to their local timezone.

FR4: Booking a slot is atomic: two concurrent requests for the same `(provider_id, slot_start)` must result in exactly one success.

FR5: A booking has a lifecycle: `[booked → completed | cancelled | no_show]`.

FR6: Cancellation is only allowed outside the provider's configured cancellation window (e.g., no cancellation within 24h).

6.3 Notifications

FR7: A reminder is sent at a configurable offset before the appointment (e.g., 24h and 1h before).

FR8: Reminder delivery is idempotent — a job retry or duplicate trigger must not send duplicate reminders.

6.4 Timezone Correctness

FR9: All slot times are stored in UTC with an explicit IANA timezone reference for display; DST transitions must not shift a booked slot's real-world time.

7. Non-Functional Requirements

Category	Requirement
Concurrency correctness	Database-level uniqueness guarantees no double-booking, verified under load test
Timezone correctness	Automated tests covering DST spring-forward/fall-back edge cases
Reliability	Reminder jobs are idempotent and retried on transient failure only

Category	Requirement
Scheduling	No app-instance-local cron; scheduled work is enqueued via a managed scheduler (§9.5)
Latency	Slot availability query p95 < 200ms for a 60-day window

8. Data Model (core entities)

`providers` (id, name, timezone, cancellation_window_hours)
`availability_rules` (id, provider_id, day_of_week, start_time, end_time)
`slots` (id, provider_id, slot_start_utc, slot_end_utc, status) — `UNIQUE(provider_id, slot_start_utc)`
`bookings` (id, slot_id UNIQUE, client_id, status, version)
`reminder_jobs` (id, booking_id, offset_type, sent_at)

The unique constraint on `slots(provider_id, slot_start_utc)` combined with a unique `bookings.slot_id` is the core double-booking guard (§7.4.1 optimistic concurrency / uniqueness).

9. Architecture Notes (per blueprint)

Next.js Server Components render availability converted to the client's browser timezone at request time; booking submission is a Client Component boundary (§6.2).

NestJS `bookings` module: booking creation is a single transactional write relying on the DB unique constraint to reject the losing concurrent request, rather than an application-level lock (§7.4.1).

Materialization of `availability_rules` into concrete `slots` runs as a scheduled job (§9.5), not on-demand, so slot IDs are stable once published.

Reminder scheduling: BullMQ delayed jobs keyed by `(booking_id, offset_type)` with a uniqueness/idempotency record to prevent duplicate sends on retry (§9.4).

All timestamps stored as UTC (`timestamp_tz` in PostgreSQL); provider/client timezone conversion happens only at the presentation layer.

10. API Surface (representative)

	
GET	<code>/providers/:id/availability?tz=America/New_York</code>
POST	<code>/bookings</code> (atomic; relies on DB uniqueness)
POST	<code>/bookings/:id/cancel</code>
GET	<code>/bookings/mine</code>

11. Milestones

Walking skeleton: one provider, static availability, manual booking, no concurrency/timezone handling yet.

Concurrent booking correctness: unique constraint + load test proving no double-booking.

Timezone correctness: DST edge-case test suite + multi-timezone display.

Reminder pipeline with idempotent delayed jobs.

Cancellation-window enforcement + full end-to-end test coverage.

12. Success Metrics (for resume/portfolio)

0 double-bookings across a load test of N concurrent requests for the same slot.

0 duplicate reminders across simulated job retries.

Documented DST edge-case test suite with pass evidence (a concrete, rare, and credible claim).

13. Open Risks

DST test coverage needs careful synthetic date construction (spring-forward gap, fall-back overlap) — treat this as a dedicated test-design task, not an afterthought.

Cross-Project Notes

All three PRDs assume the same base repository setup from the blueprint (§20 Project Setup Checklist): Turborepo + pnpm workspace, shared TypeScript/lint config, CI running lint/typecheck/test/build, and a documented ADR for any deviation from the default stack. Building them in sequence (Invoicing → Marketplace → Scheduling) lets you reuse and mature the same repository conventions, auth patterns, and observability setup across all three — which is itself a strong resume point ("built and operated three production-shaped services on one shared internal platform standard").